



PONTIFICIA UNIVERSIDAD CATOLICA DE CHILE
ESCUELA DE INGENIERIA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACION

Diseño y Análisis de Algoritmos - IIC2283 - 2^{do} semestre - 2025

Tarea 2

Algoritmos Codiciosos y Transformada rápida de Fourier

Publicación : Martes 14 de octubre
Github Classroom : https://classroom.github.com/a/b1h_xb7s
Entrega : Jueves 30 de octubre

Indicaciones

- La tarea es estrictamente individual.
- La solución debe ser entregada en los archivos `t1p1.py` y `t1p2.py` del repositorio privado asignado mediante GitHub Classroom para esta tarea. Se revisará el último *commit* subido antes de la entrega al repositorio. Se usará Python 3.10.X para la revisión.
- El *input* para el programa debe ser obtenido desde *standard input*. El *output* debe ser entregado mediante *standard output*.
- La corrección se realizará mediante *tests* automatizados acordes al formato de *input* y *output* especificado. Cada *test* tendrá un timeout según lo que se especifica como tiempo esperado.
- Un *test* se considerará **reprobado** en caso de que 1) dado el *input* el *output* sea incorrecto, 2) exista un error de *runtime* durante su ejecución, o 3) el *timeout* se cumpla durante su ejecución. En otro caso, el *test* se considerará **aprobado**.
- No se permite el uso de librerías externas a la librería estándar de Python *a priori*. Consultar en las [issues del repositorio oficial del curso](#) en caso de requerir una.
- A 5 días de la fecha final del plazo, se publicará un test de gran tamaño para cada parte, siendo el único test que publicaremos. La habilidad de generar tests es parte de la evaluación.
- Para esta tarea aplica la política de atrasos descrita en el programa del curso.
- **NO** se garantiza la existencia de una nueva instancia de recorreción. Deben asegurarse de entregar el output **solo** por *standard output* y **no** imprimir el tiempo.

1. La palabra de Jackenstein (50 %)

Jackenstein es torpe y muy miedoso, pero si hay algo que lo aterra es la luz y los strings lexicográficamente pequeños.



Figura 1: Jackenstein sin saber que lo vamos a asustar

Afortunadamente, se acerca Halloween y tienes guardados n strings para esta ocasión, por lo que decides concatenar todos los strings en algún orden tal que provoque el mayor terror a Jackenstein posible.

Determina cuál es el string lexicográficamente menor que puede resultar de concatenar los n strings.

Input

La entrada consiste en varias líneas. La primera con un entero n , la cantidad de strings ($1 \leq n \leq 10^4$).

Cada una de las siguientes n líneas contiene un string s_i compuesto únicamente por letras minúsculas del alfabeto inglés. Se asegura que cada string cumple que $|s_i| \leq 50$.

Output

La salida debe contener un único string correspondiente al string lexicográficamente menor al concatenar los n strings.

Tiempo límite

Se espera que la solución se ejecute en un tiempo **menor o igual a 2 segundos** para cualquier instancia de input según las restricciones dadas.

Complejidad esperada

Se espera que la solución posea una complejidad de $O(nL \log(n))$, donde L es el tamaño del string más grande entre los n strings del input.

Ejemplos

Input de ejemplo 1
4 abba abacaba bcd er
Output de ejemplo 1
abacabaabbabcbder

Input de ejemplo 2
5 x xx xxa xxaa xxaaa
Output de ejemplo 2
xxaaaxxaaxxxxx

2. Profesores escalofriantes (50 %)

Se aproxima Halloween en el DCC y los célebres profesores del momento Diego y Juan lo celebrarán repartiendo dulces a sus alumnos. No obstante, las reglas entre los profesores son claras: deben entregar la misma cantidad de dulces en total a sus respectivas secciones.

Para esto ambos profesores acordaron que solo podrían utilizar k tipos distintos de puñados de dulces, cuyas cantidades eran innegociables: $d_1, d_2, \dots, d_k$. Cada profesor le entregará a cada alumno de su sección exactamente un puñado de dulces, elegido de entre estas k cantidades preacordadas.

Sabiendo que cada sección posee $\frac{n}{2}$ alumnos, con n siendo el número total de alumnos (siempre par), los profesores se preguntan: ¿cuántas posibles asignaciones distintas alumno-dulces existen tal que la cantidad de dulces entregada en la sección de Diego sea exactamente igual a la cantidad de total entregada en la sección de Juan?

Dos asignaciones se dicen distintas si existe al menos un alumno x que recibe una cantidad de dulces diferente en una asignación respecto a la otra.

Debido a que el número de asignaciones puede ser muy elevado, la respuesta debe ser imprimida en consola módulo **998244353**.

Input

La primera línea de la entrada consiste de dos enteros: n y k , la cantidad de alumnos totales y el número de puñados disponibles respectivamente, se asegura que n es un número par ($2 \leq n \leq 5 \cdot 10^3, 1 \leq k \leq 10$).

A continuación, sigue la segunda línea con k enteros $d_1, d_2, \dots, d_k$, las cantidades de dulces por puñados que utilizarán los profesores. Se asegura que todos los puñados d_i son distintos ($0 \leq d_i \leq 9$)

Output

La salida contiene un único entero, la cantidad de asignaciones distintas modulo 998244353 tal que ambas secciones reciban la misma cantidad dulces.

Tiempo límite

Se espera que la solución se ejecute en un tiempo **menor o igual a 4 segundos** para cualquier instancia de input según las restricciones dadas.

Complejidad esperada

Se espera que la solución posea una complejidad de $O(n \log n)$, aunque una complejidad $O(n \log^2 n)$ también es suficiente bajo los límites del problema.

Hints

Recomendamos primero formular el problema con una solución de programación dinámica y luego optimizar esa solución utilizando transformada rápida de fourier.

Para manejar la parte del modulo en la respuesta, recomendamos investigar sobre la Numeric Theoretic Transform ([NTT](#)), aunque existen otros metodos que de todas formas les permitiran llegar a la respuesta.

Ejemplo

Input de ejemplo 1
4 2
1 8
Output de ejemplo 1
6

Input de ejemplo 2
20 1
6
Output de ejemplo 2
1

Input de ejemplo 3
1000 7
5 4 0 1 8 3 2
Output de ejemplo 3
460571165

Explicación de los casos

En el primer caso de ejemplo hay 4 alumnos en total y por tanto, 2 alumnos en cada sección. Tenemos 6 posibles asignaciones (secciones separadas por un -):

1. $\{1, 1\} - \{1, 1\}$
2. $\{1, 8\} - \{1, 8\}$
3. $\{1, 8\} - \{8, 1\}$
4. $\{8, 1\} - \{8, 1\}$
5. $\{8, 1\} - \{1, 8\}$
6. $\{8, 8\} - \{8, 8\}$

En el segundo caso de prueba tenemos 20 alumnos y un solo tipo de puñado, la única asignación posible es asignar a todos los alumnos 6 dulces y por tanto, ambos profesores repartieron la misma cantidad de dulces en cada sección.

Política de integridad académica

Los/as estudiantes de la Escuela de Ingeniería de la Pontificia Universidad Católica de Chile deben mantener un comportamiento acorde a la Declaración de Principios de la Universidad. En particular, se espera que mantengan altos estándares de honestidad académica. Cualquier acto deshonesto o fraude académico está prohibido; los/as estudiantes que incurran en este tipo de acciones se exponen a un Procedimiento Sumario. Es responsabilidad de cada estudiante conocer y respetar el documento sobre Integridad Académica publicado por la Dirección de Docencia de la Escuela de Ingeniería.

Específicamente, para los cursos del Departamento de Ciencia de la Computación, rige obligatoriamente la siguiente política de integridad académica. Todo trabajo presentado por un/a estudiante para los efectos de la evaluación de un curso debe ser hecho individualmente por el/la estudiante, sin apoyo en material de terceros. Por “trabajo” se entiende en general las interrogaciones escritas, las tareas de programación u otras, los trabajos de laboratorio, los proyectos, el examen, entre otros.

En particular, si un/a estudiante copia un trabajo, o si a un/a estudiante se le prueba que compró o intentó comprar un trabajo, obtendrá nota final 1.1 en el curso y se solicitará a la Dirección de Pregrado de la Escuela de Ingeniería que no le permita retirar el curso de la carga académica semestral.

Por “copia” se entiende incluir en el trabajo presentado como propio, partes hechas por otra persona. En caso de que corresponda a “copia” a otros estudiantes, la sanción anterior se aplicará a todos los involucrados. En todos los casos, se informará a la Dirección de Pregrado de la Escuela de Ingeniería para que tome sanciones adicionales si lo estima conveniente.

También se entiende por copia extraer contenido sin modificarlo sustancialmente desde fuentes digitales como Wikipedia o mediante el uso de asistentes inteligentes como ChatGPT o Copilot. Se entiende que una modificación sustancial involucra el análisis crítico de la información extraída y en consecuencia todas las modificaciones y mejoras que de este análisis se desprendan. Cualquiera sea el caso, el uso de fuentes bibliográficas, digitales o asistentes debe declararse de forma explícita, y debe indicarse cómo el/la estudiante mejoró la información extraída para cumplir con los objetivos de la actividad evaluativa.

Obviamente, está permitido usar material disponible públicamente, por ejemplo, libros o contenidos tomados de Internet, siempre y cuando se incluya la referencia correspondiente.

Lo anterior se entiende como complemento al Reglamento del Estudiante de la Pontificia Universidad Católica de Chile (<https://registrosacademicos.uc.cl/reglamentos/estudiantiles/>). Por ello, es posible pedir a la Universidad la aplicación de sanciones adicionales especificadas en dicho reglamento.

Compromiso del Código de Honor

Este curso suscribe el Código de Honor establecido por la Universidad, el que es vinculante. Todo trabajo evaluado en este curso debe ser propio. En caso de que exista colaboración permitida con otros/as estudiantes, el trabajo deberá referenciar y atribuir correctamente dicha contribución a quien corresponda. Como estudiante es un deber conocer el Código de Honor (<https://www.uc.cl/codigo-de-honor/>)